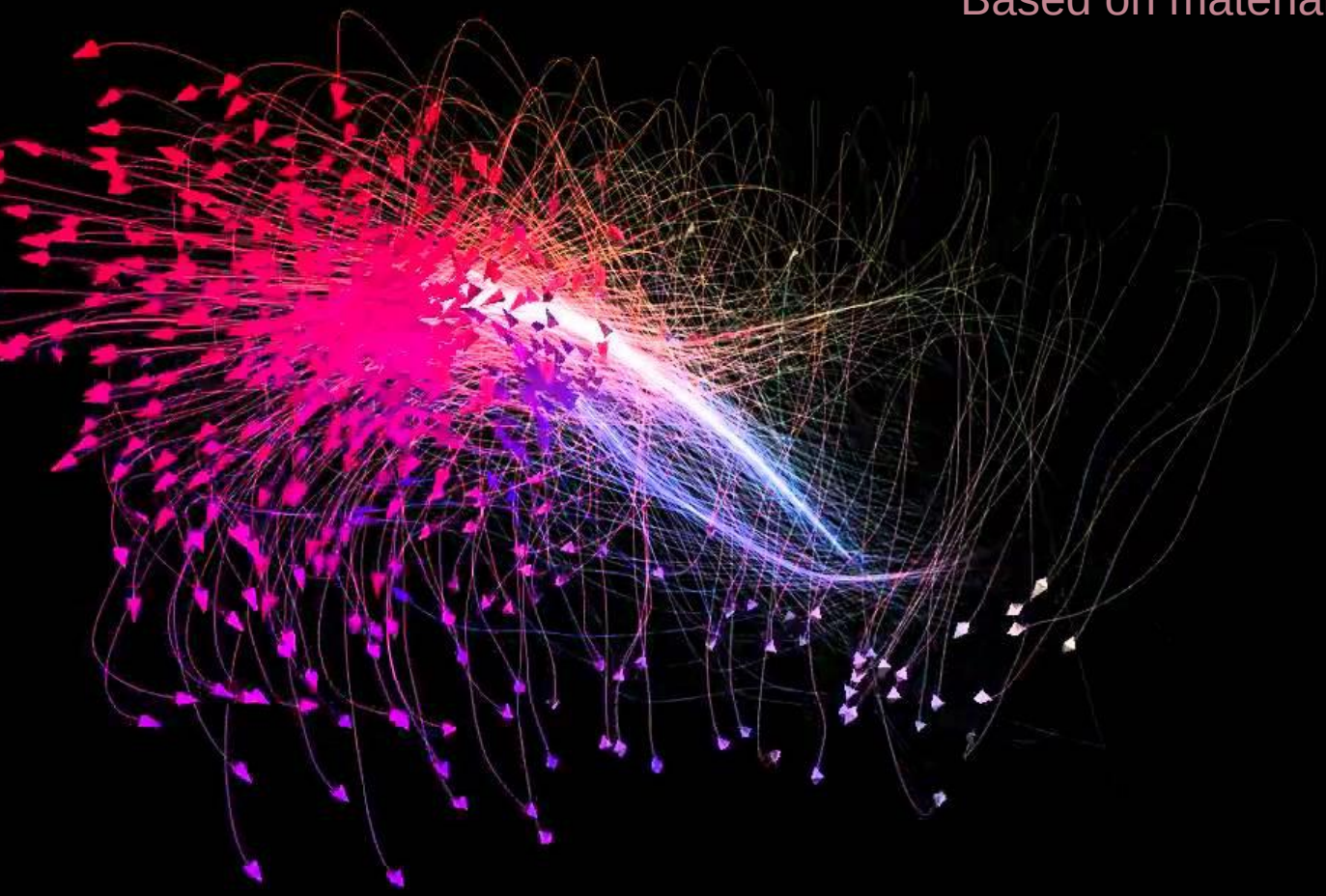
Computational Intelligence
Local Search

Based on material from Harry Goldingay



Anikó Ekárt & Alina Patelli

Aston University

Outline

- At the end of this lecture you should…
  - …be able to explain why it is important to have more than one solution to a problem
  - …understand the principles of designing a local search algorithm
  - …understand two given discrete and real valued local search algorithms well enough to implement them



Motivation



Is This Fast?

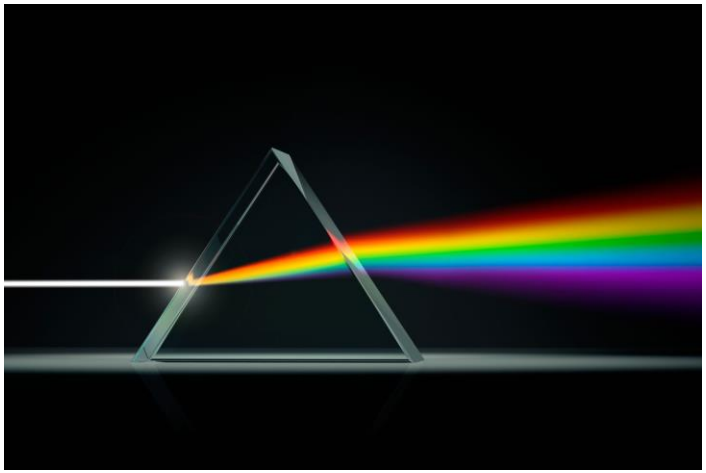


Is This Fastest?









Motivation

- In this module, you will:
  - learn some **well known** Computational Intelligence techniques,
  - learn enough to **design your own**

- How do you know that a problem instance is difficult enough to require thought?

- How will you know whether your algorithm is a good way to solve a problem?
  - Compare to the **best possible solution**?
  - **Empirical comparisons**:
    - to **common/simple heuristics**?
    - to other **Computational Intelligence techniques**?
    - to the **state-of-the-art**?



Motivation

- In this lecture we will introduce some simple heuristics for optimisation problems:
  - Good **“first guess”** solution methods
    - very easy to implement
    - will solve simple problems well
  - Good **baselines** for other algorithms
    - if your algorithm performs worse than random search on a problem, you need to rethink it!
  - **Synergies** with computational intelligence algorithms
    - can provide good starting points…
    - …or improve final solution quality



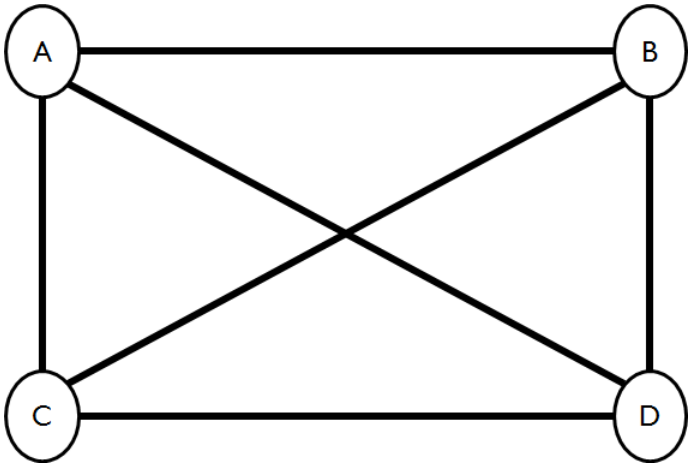
What is an Optimization Problem?



Recap

- Yesterday two problems were introduced:

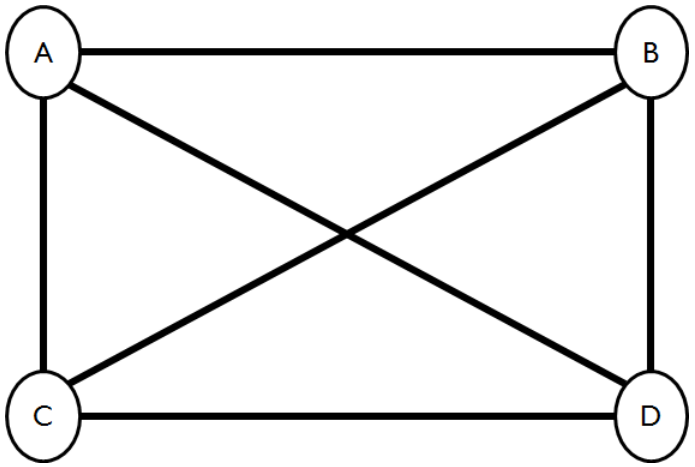




- How are they similar?

Recap





- We aim to extremize some quantity…
- …by choosing a set of parameters…
- …under some constraints

- **Both are optimization problems**

Optimization Terminology

Term	Definition	TSP Example
Dimension	The size of the vector necessary to represent a solution	The tour length is the same as the number of cities
Search Space	The range of values which it is possible for a solution to take	A tour can't contain a city which doesn't exist!
Constraints	Further restrictions on the relationships between our variables	A city can only appear once in the tour
Feasible Region	The subset of the search space which meets the constraints	
Cost/objective/ fitness function	A function which tells us the quality of our solution. Our aim is to min/maximise	The length of the tour



Optimization Aims

- Formally:
  - To find a solution (the best possible choice),
  - To prove what you have found is a solution,
  - Or to prove that no solution exists
- Obstacles:
  - No known polynomial time algorithms for many problems,
  - Intuition is that such algorithms don't exist (P≠NP)
- In practice:
  - to find ***a solution of sufficient quality***…
  - …with the resources available (primarily, ***time***)



Constructive Heuristics for the TSP



Search Paradigms

- Two search paradigms for combinatorial problems:
  - **Constructive**
    - Build a solution from scratch (e.g. by combining partial solutions)
  - **Perturbative**
    - Start with some (suboptimal) solution
    - Make small modification (perturbations) repeatedly to improve it

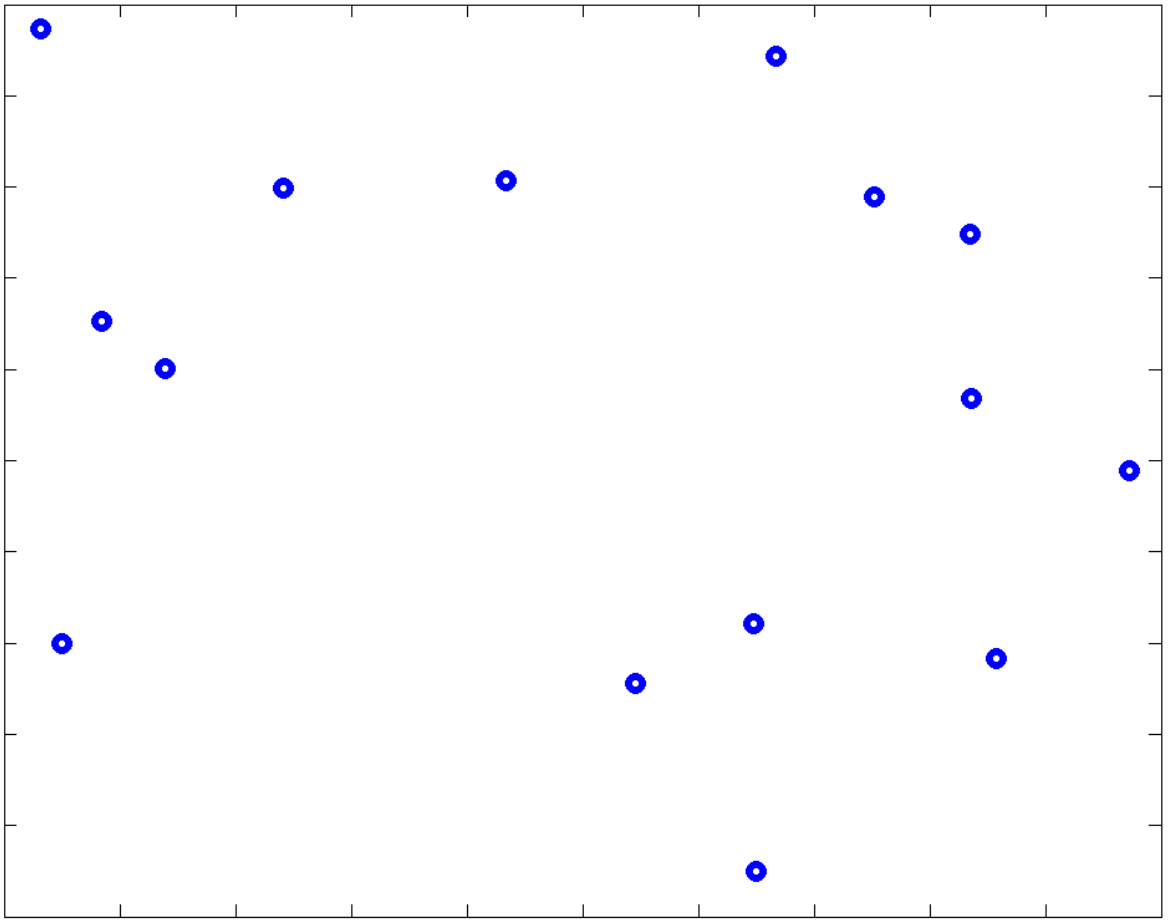


TSP: Solution Approaches

- What are your ideas for solving the TSP?



Example TSP Instance

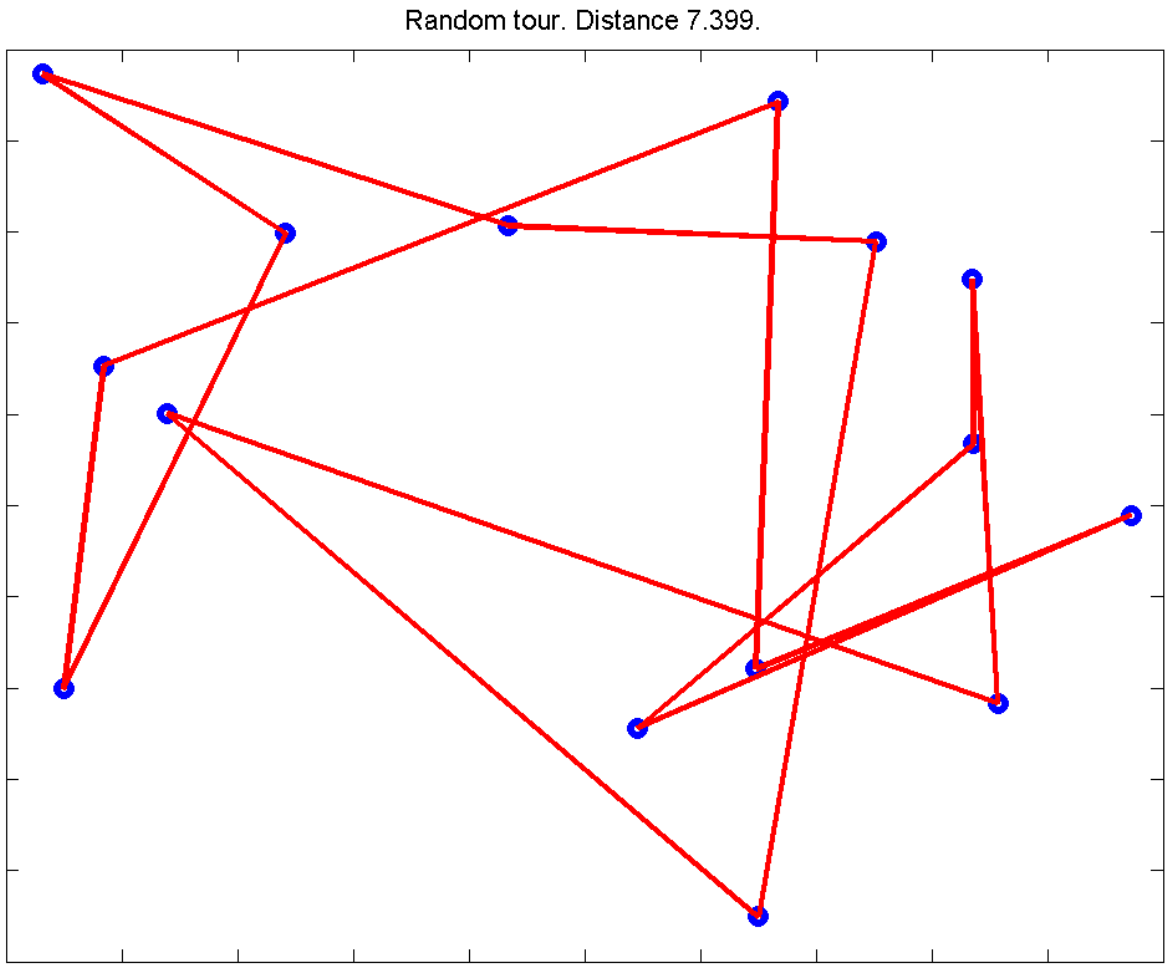


TSP: Solution Approaches

- What are your ideas for solving the TSP?
- Random Search



Random Tour



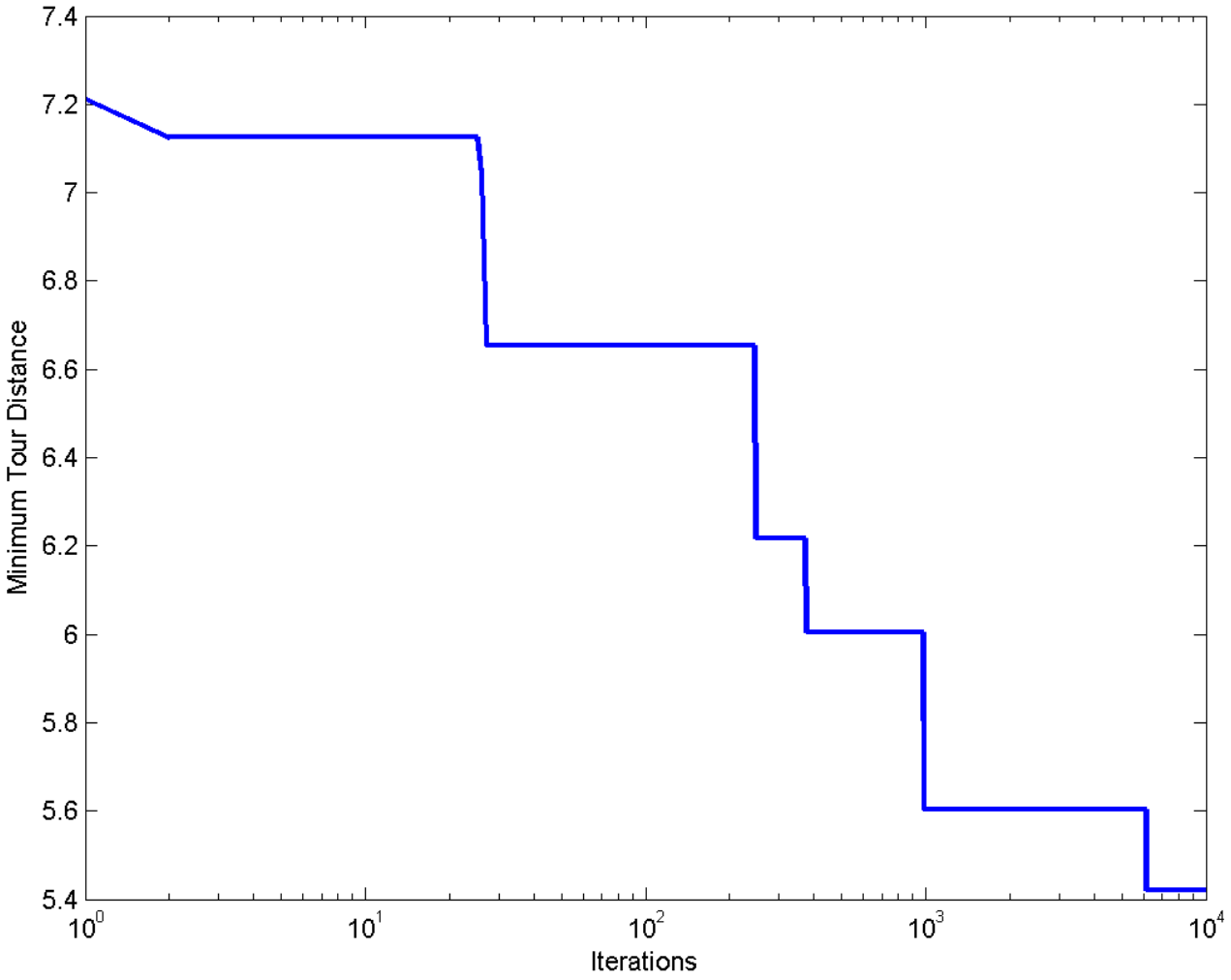
TSP: Solution Approaches

- What are your ideas for solving the TSP?

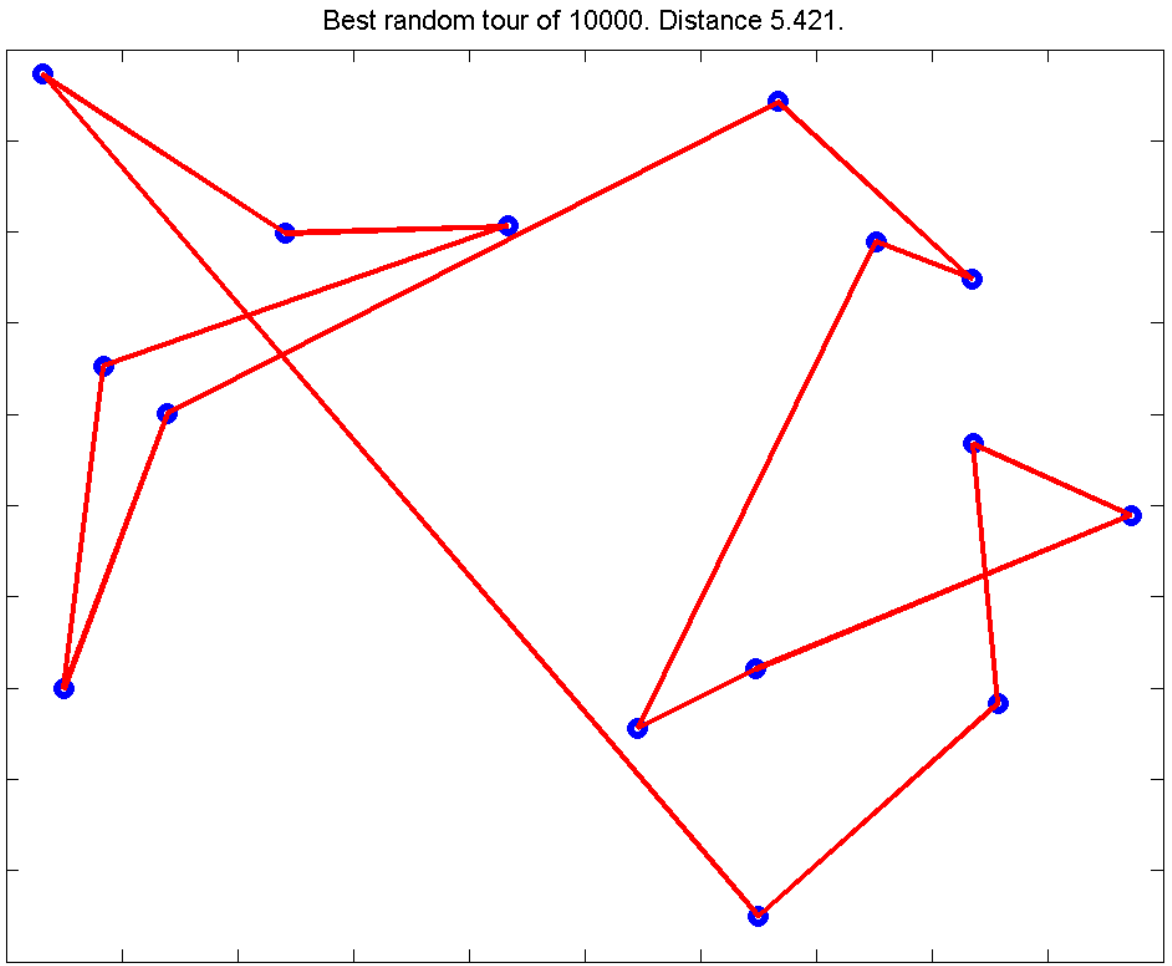
- Random search
- Repeated random search



Repeated Random Tours



Best Random Tour



TSP: Solution Approaches

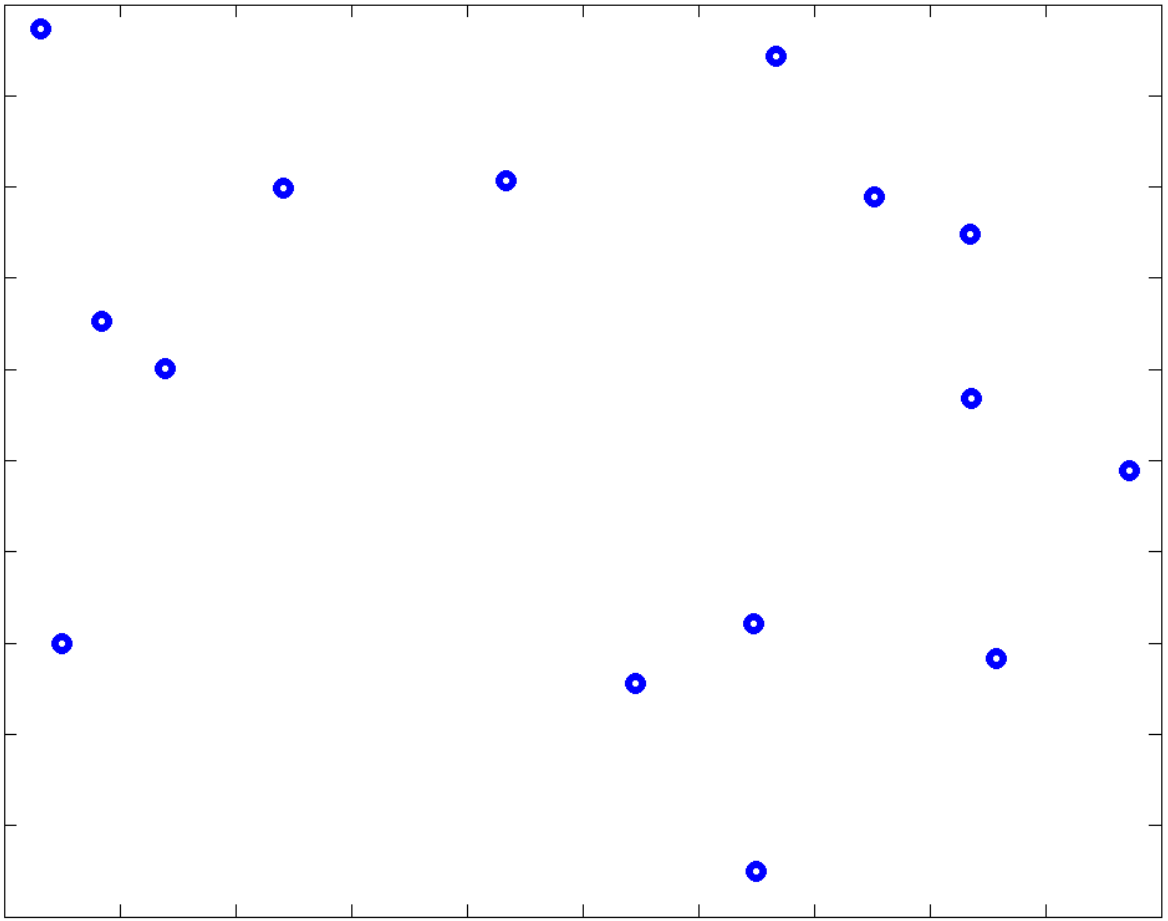
- What are your ideas for solving the TSP?

- Random search
- Repeated random search

- Still not great… Any other ways of constructing a solution?



Example TSP Instance



Greedy Algorithms

- **Greedy algorithms** make locally optimal decisions without thought for the long-term consequences

- Advantages:
  - Fast! Easy to design (low degree) polynomial algorithms.
  - Guarantee that good solution components are used

- Disadvantages:
  - In a constrained setting, good initial decisions can force you into bad decisions later!

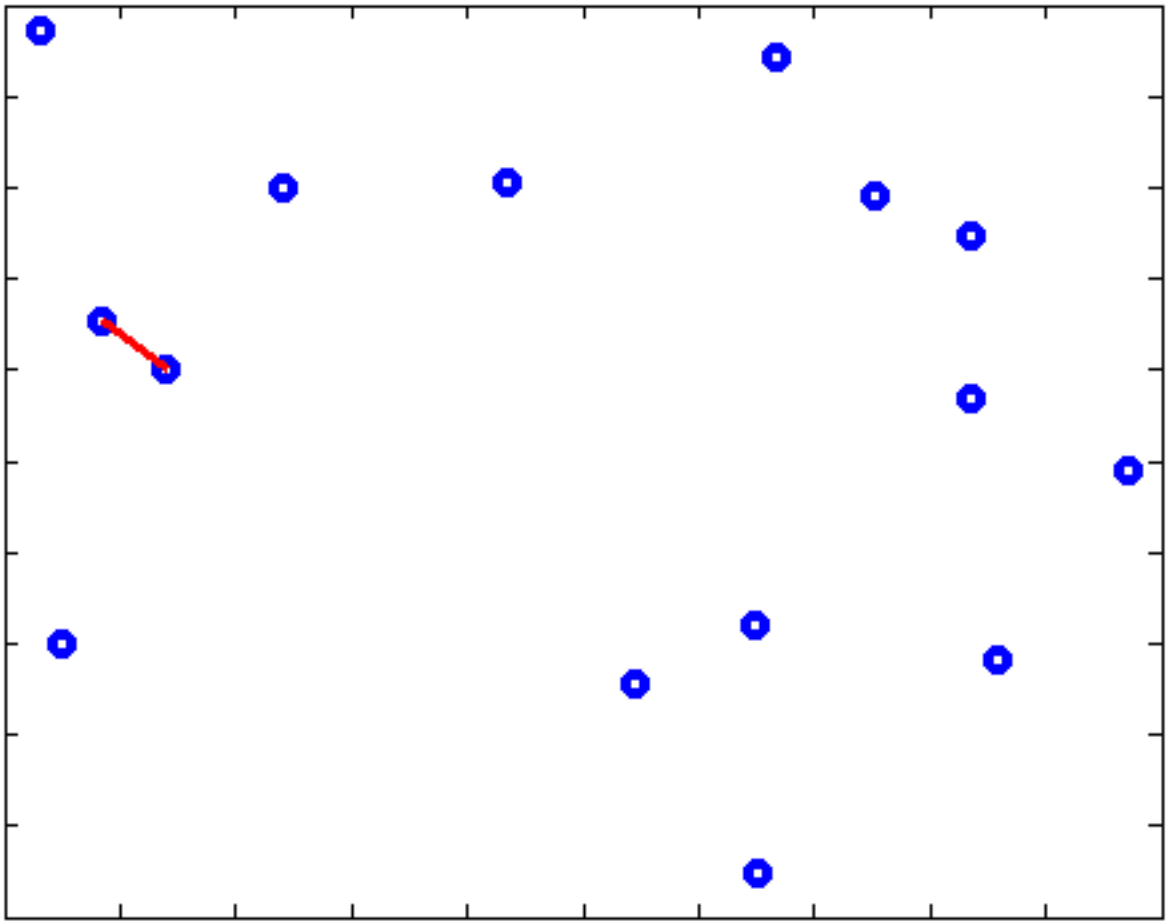


A Greedy Algorithm for the TSP

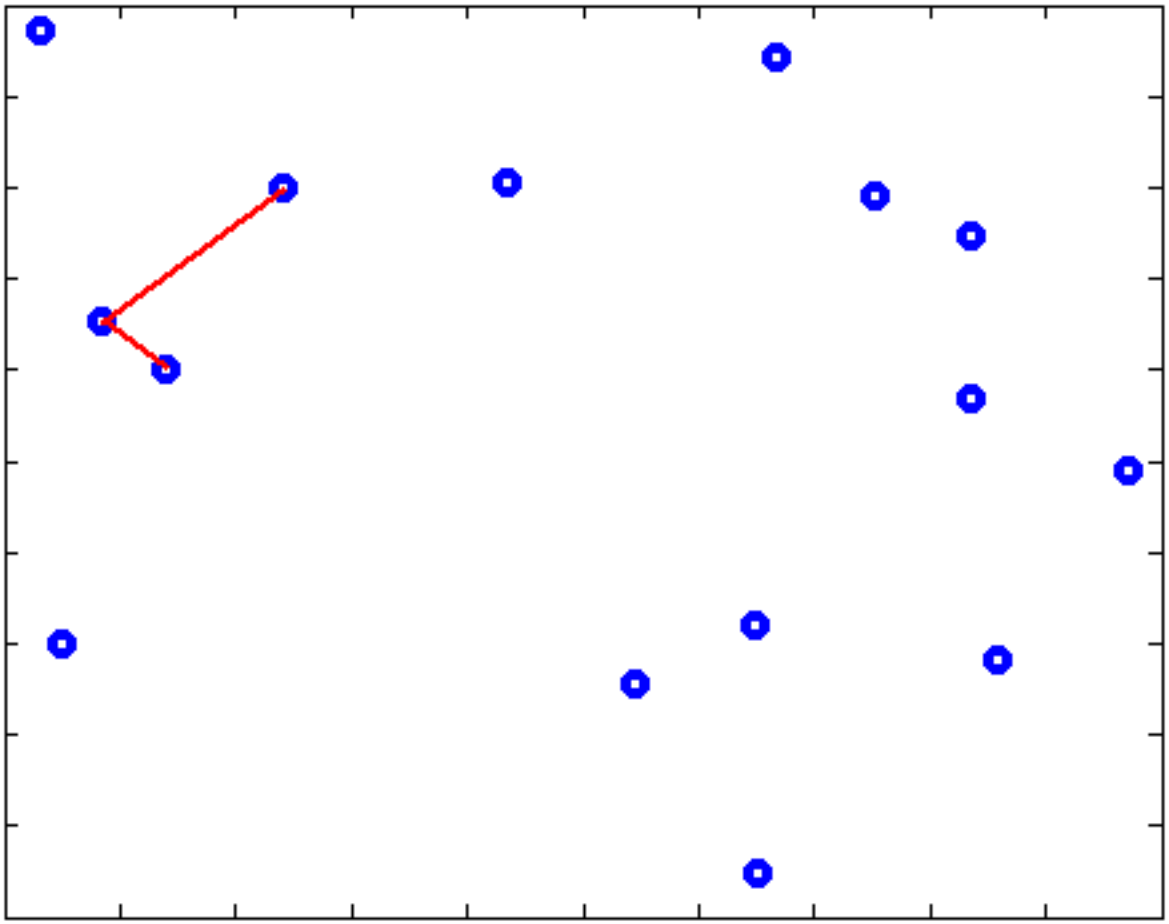
- Find the two cities with the **smallest distance** between them
  - Make one the start of the tour
  - Make the other the end of the tour
- Until all cities are in the tour
  - Find the **closest city** to the city at either end of the tour
  - Add this city to the appropriate end of the tour
- **Note:** not exactly “greedy” – some choices are suboptimal – but simple illustration



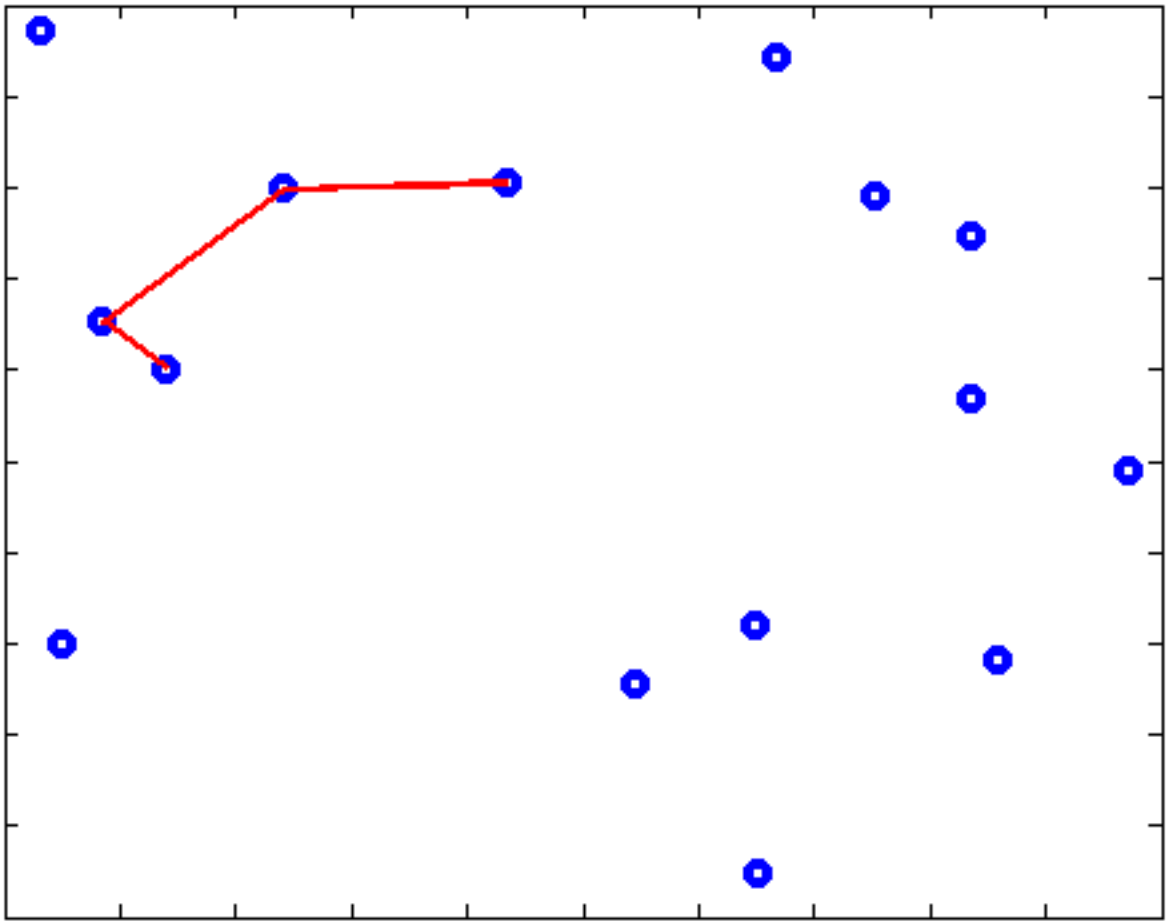
Greedy Tour Construction (0)



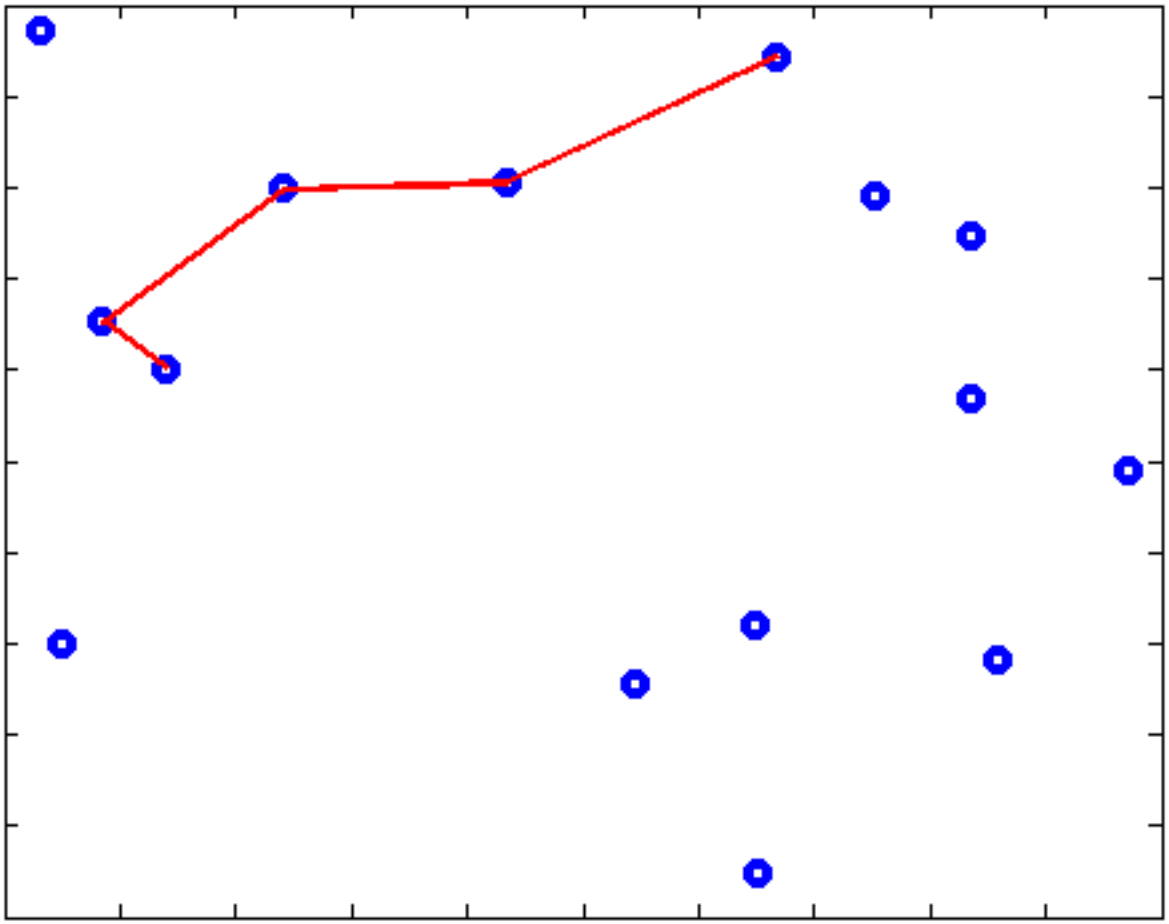
Greedy Tour Construction (1)



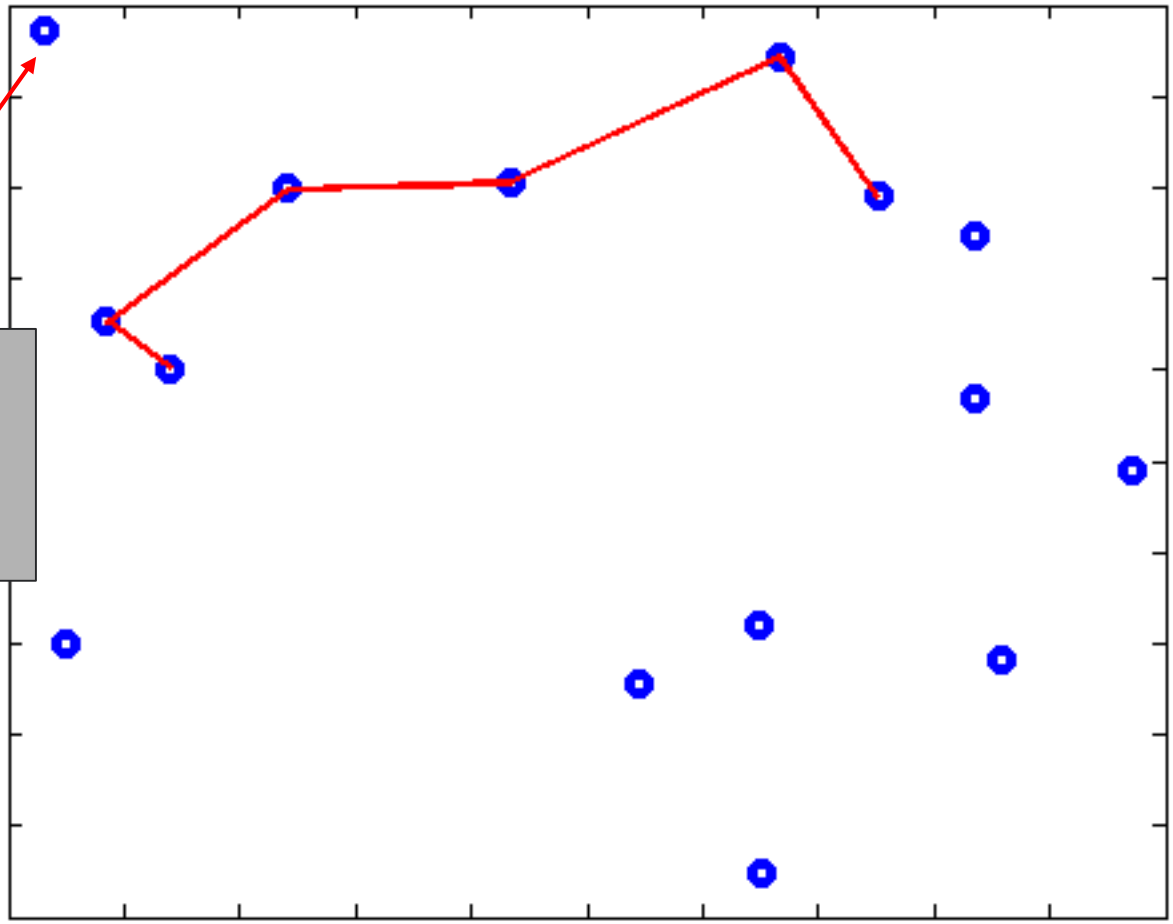
Greedy Tour Construction (2)



Greedy Tour Construction (3)

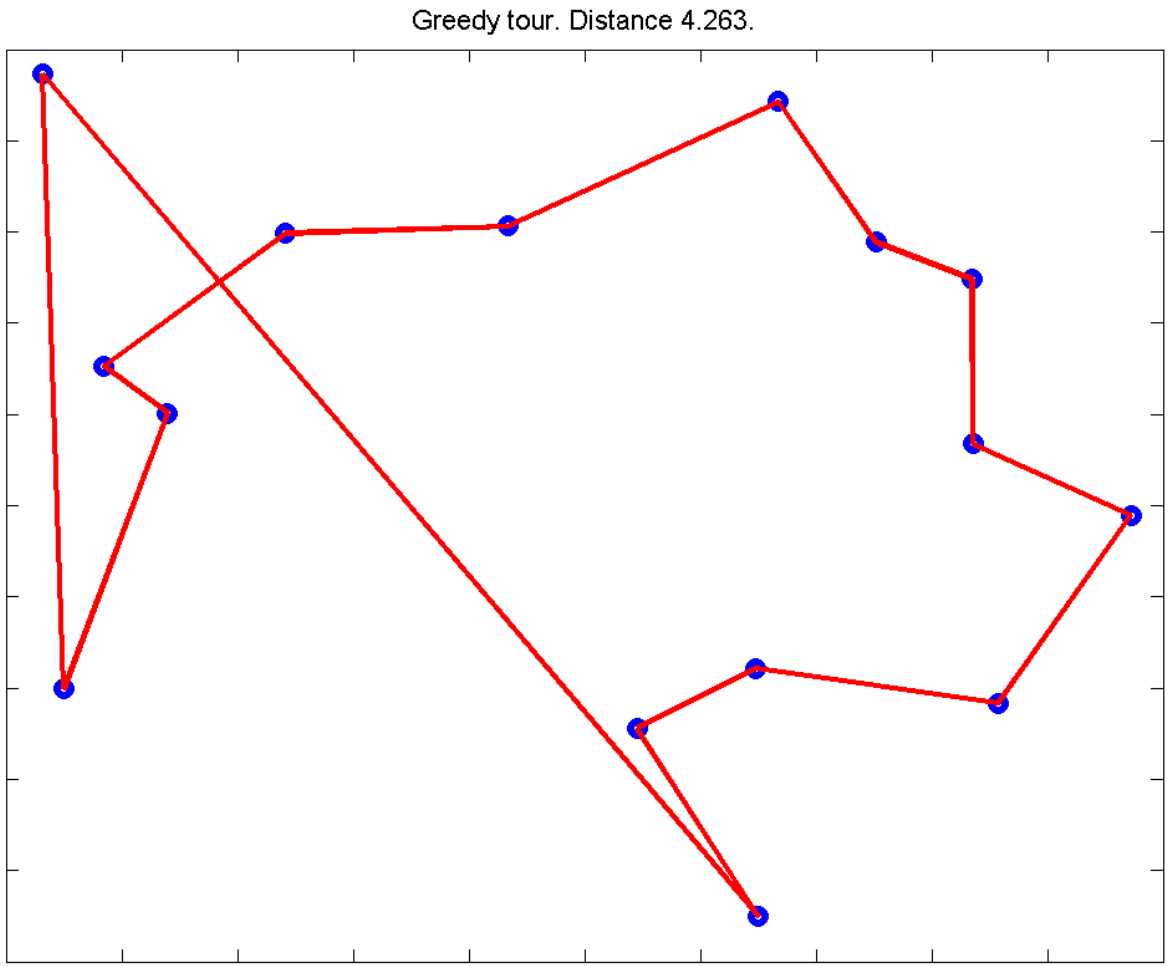


Greedy Tour Construction (4)



Looks good so far, but we are storing up trouble for later!

Greedy Tour

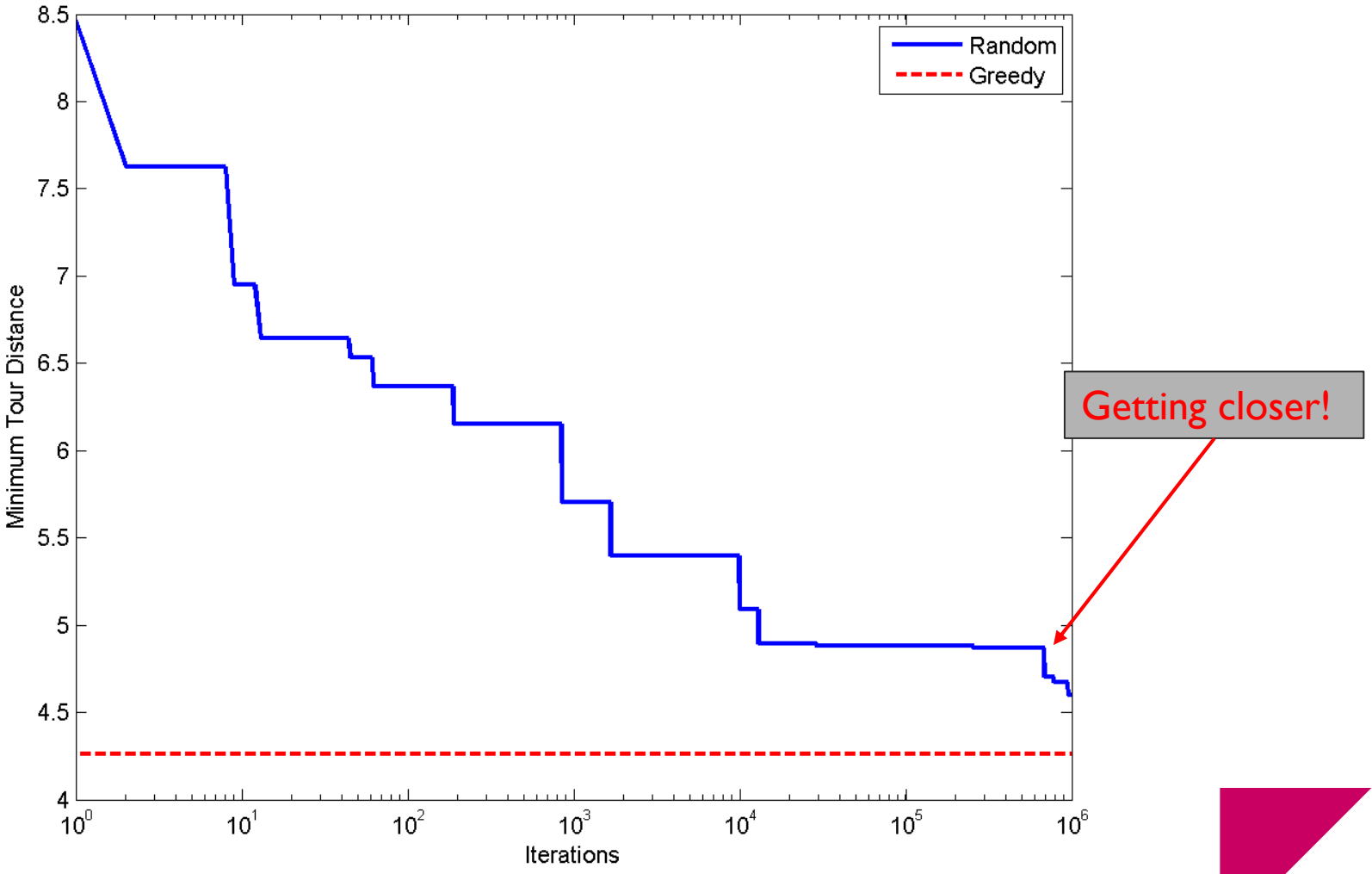


Random vs Greedy

- In this example, the greedy algorithm outperformed random search
  - Greedy tour length: 4.263
  - Best random tour length: 5.421
- Does this mean that the greedy algorithm is better?



Repeated Random Tours vs Greedy Tour



Random vs Greedy

- In this example, the greedy algorithm outperformed random search. Does this mean that it is better?

- Depends how much time you have!
  - The greedy algorithm only generates **one solution**
  - This solution is (for interesting problems) **suboptimal**
  - Random search will **catch up eventually**, but possibly after an infeasibly long time

- Are constructive heuristics the only approach?



Perturbative Heuristics

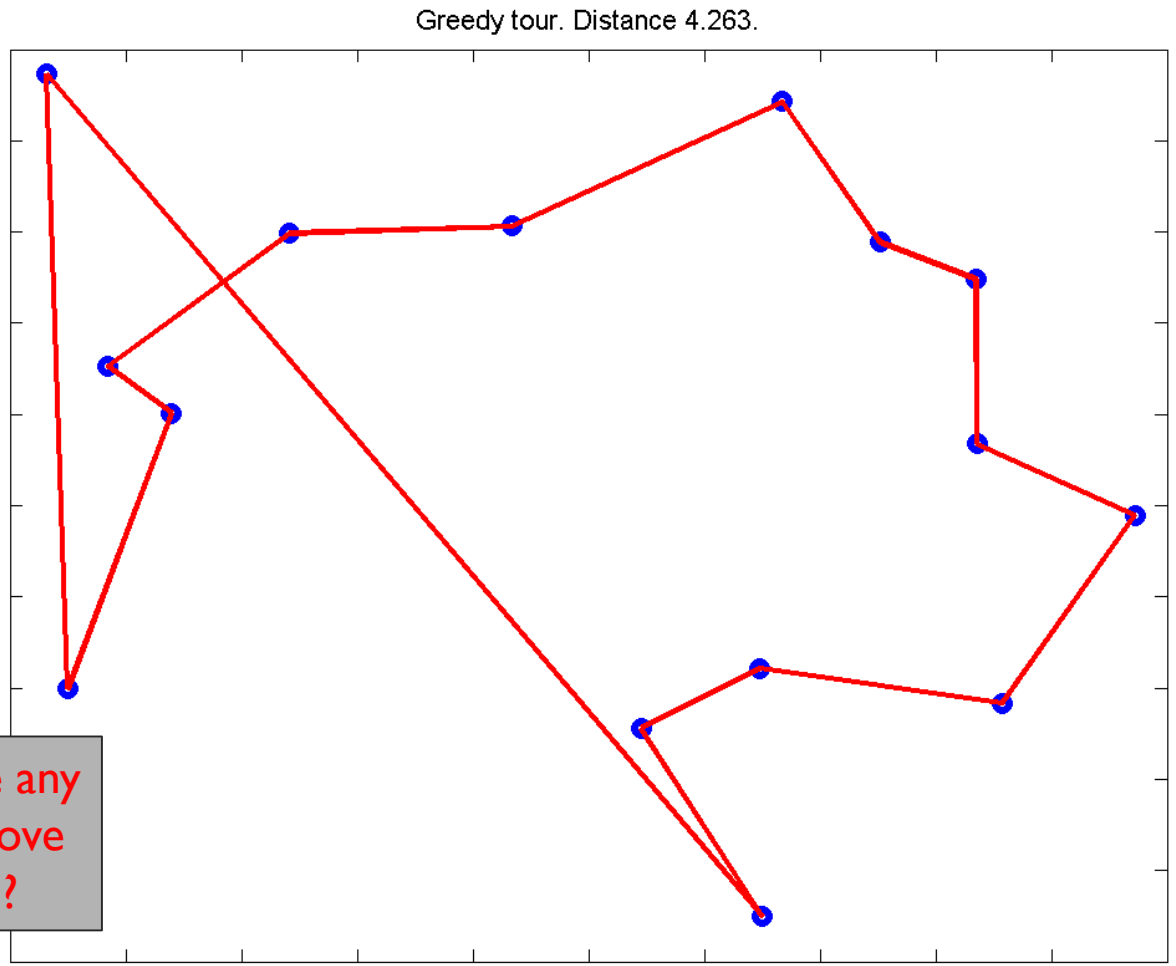


Search Paradigms

- Two search paradigms for combinatorial problems:
  - **Constructive**
    - Build a solution from scratch (e.g. by combining partial solutions)
  - **Perturbative**
    - Start with some (suboptimal) solution
    - Make small modification (perturbations) repeatedly to improve it
- The two search methods we have seen (random and greedy) are **constructive**
- Why and how would we use **perturbative search**?



Greedy Tour Revisited

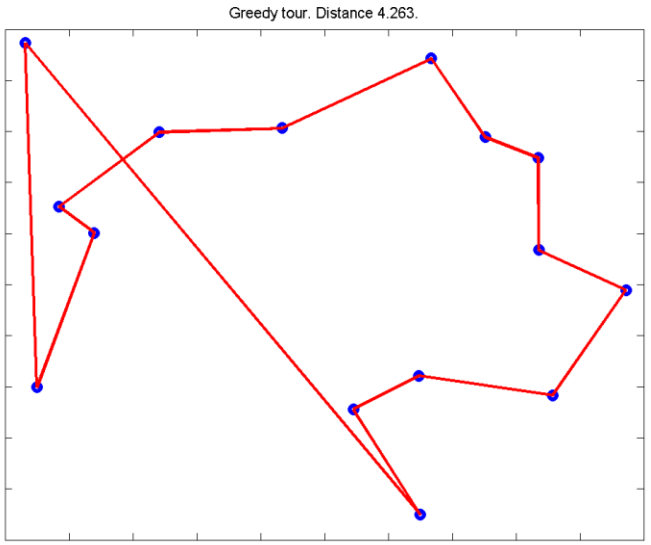


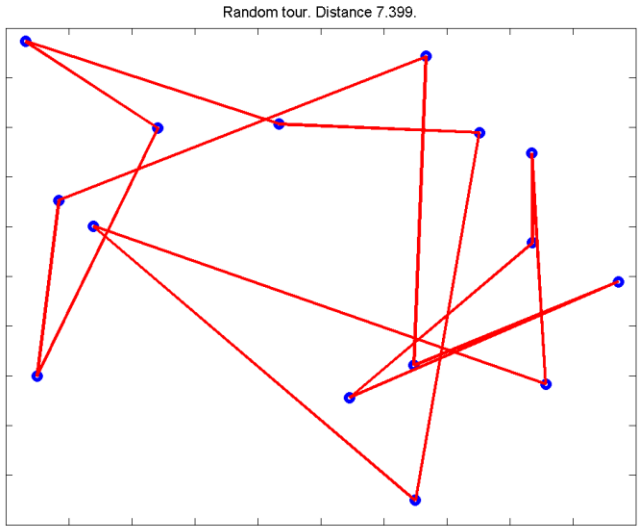
Can you see any way to improve this solution?

Local Search: Motivation

- Some aspects of the greedy solution are sub-optimal…
- …but some are good

- Would we be better off looking at **similar solutions** to the greedy tour…
- …or going back to a random search of the space?






HOT
or
COLD

Hot or Cold Game

- **The Rules**:
  - Start at the door
  - Take some steps in any direction and stop
  - The class will give you feedback on whether you are closer to your objective (**hotter** or **colder**)
  - Keep taking steps until you have found your objective

- What would your strategy to win this game in the minimum number of steps be?

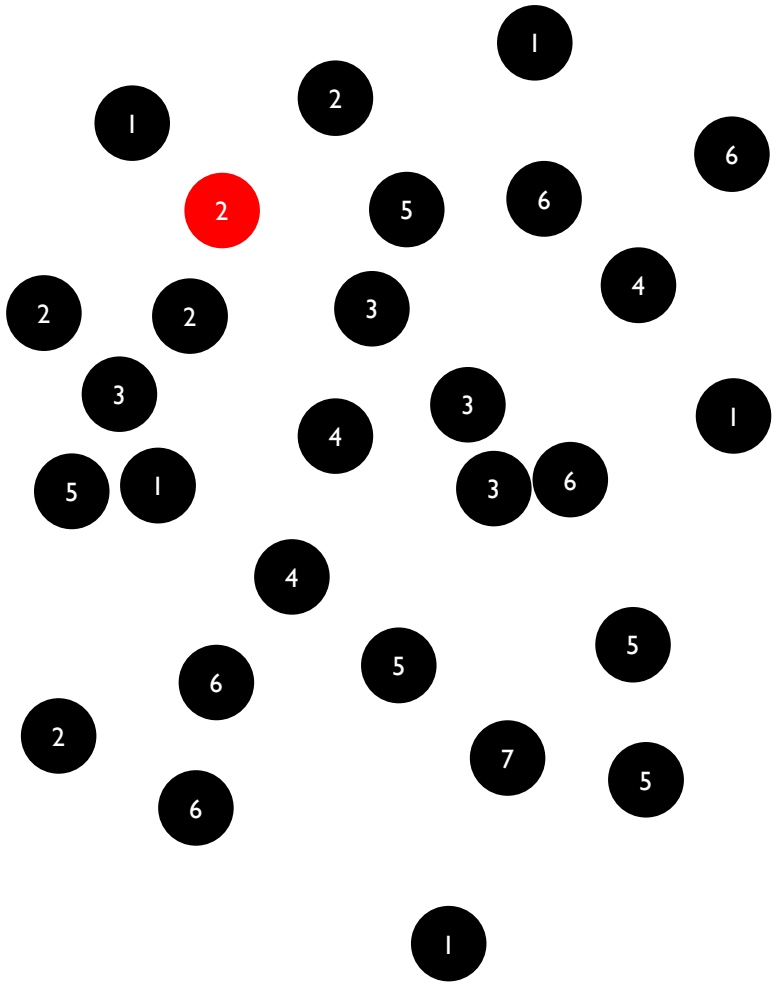


Hot or Cold Game: Conclusions

- We *can* use local information (**hotter** or **colder**) to solve a global problem (where is the objective?)
- However, local information can also be misleading – we can end up stuck in a **local optimum**
- Can think of what we have been doing as a form of **local search**

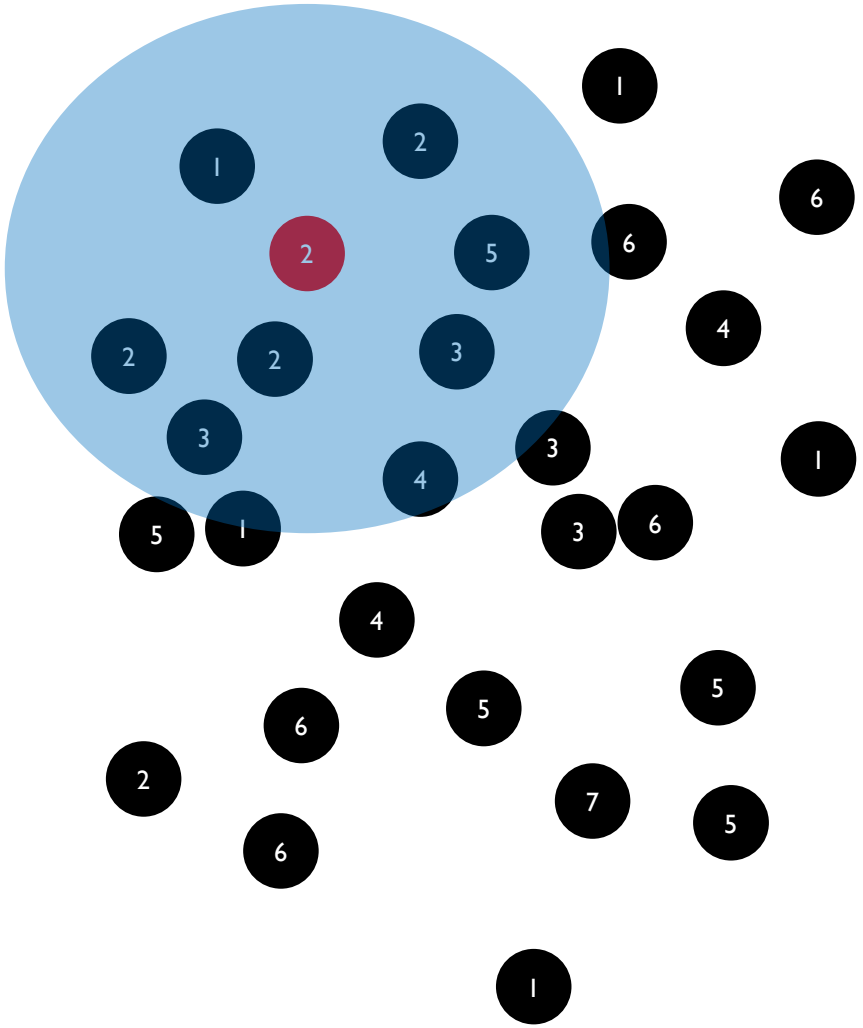


Local Search Illustrated



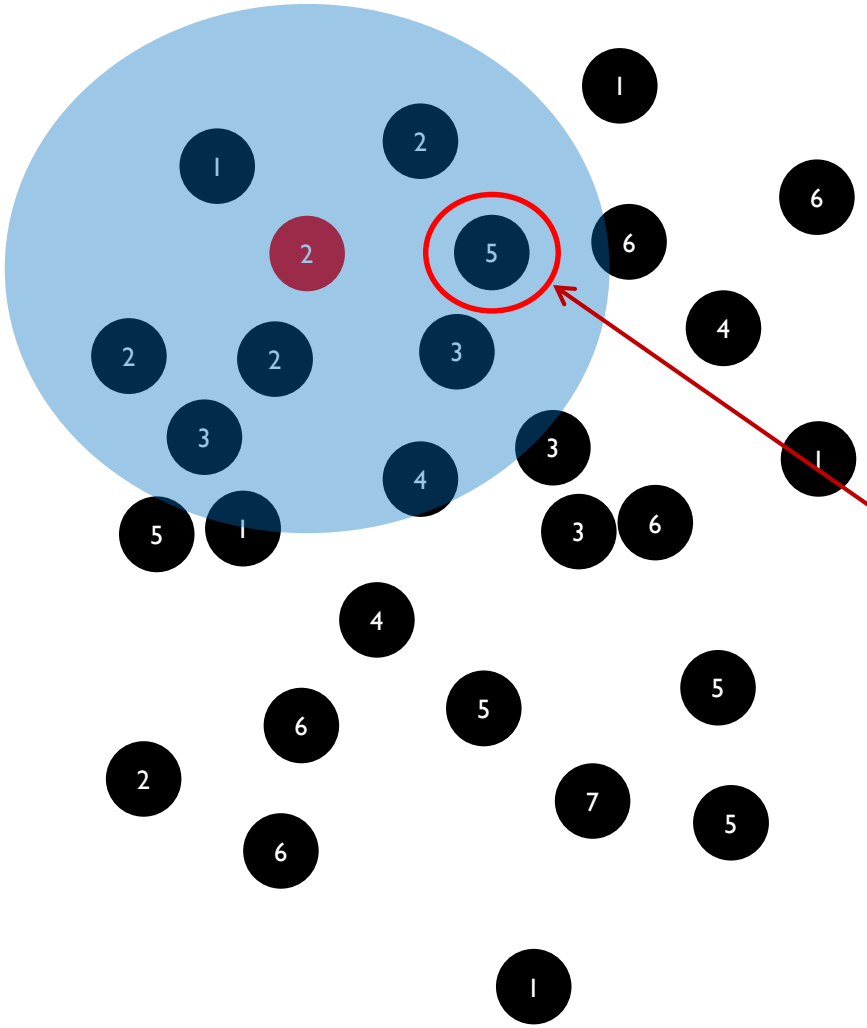
- Start at a fixed solution.

Local Search Illustrated



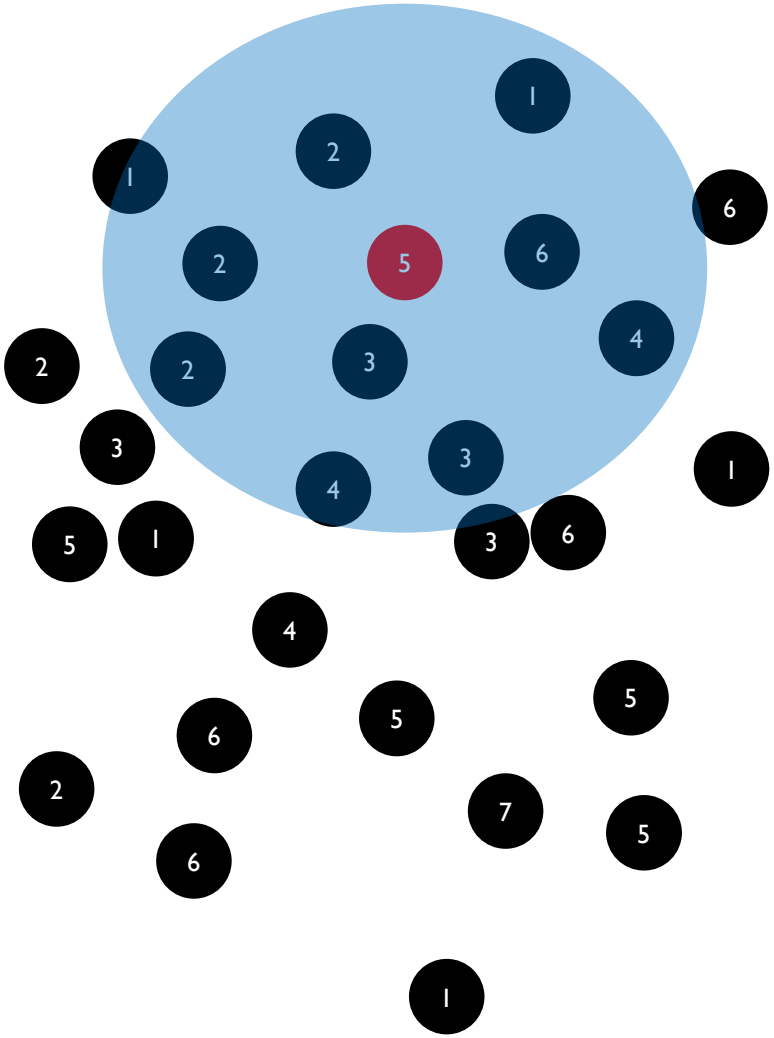
- Start at a fixed solution
- Take some “neighbourhood” of close solutions

Local Search Illustrated



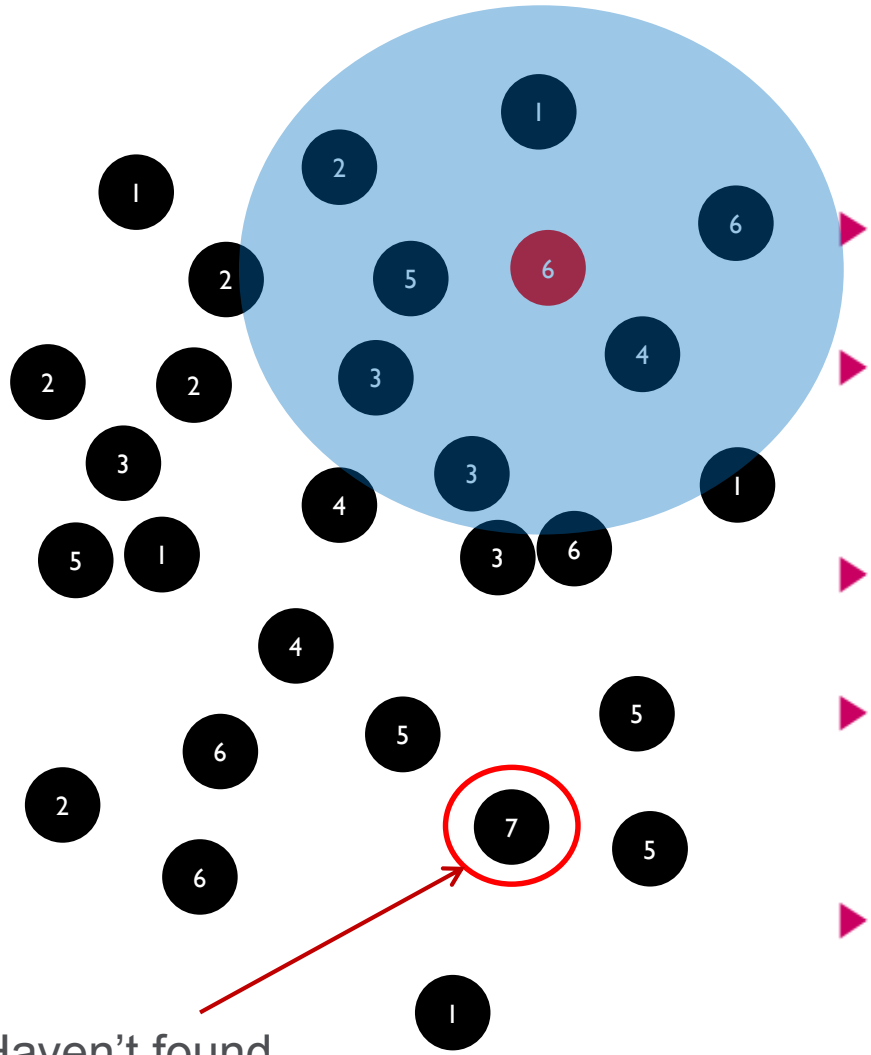
- Start at a fixed solution
- Take some "neighbourhood" of close solutions
- Find the best neighbour…

Local Search Illustrated



- Start at a fixed solution
- Take some “neighbourhood” of close solutions
- Find the best neighbour…
- …and continue the search from there

Local Search Illustrated



- Start at a fixed solution
- Take some "neighbourhood" of close solutions
- Find the best neighbour…
- …and continue the search from there
- Stop when no further improvement possible (or some other **termination condition**)

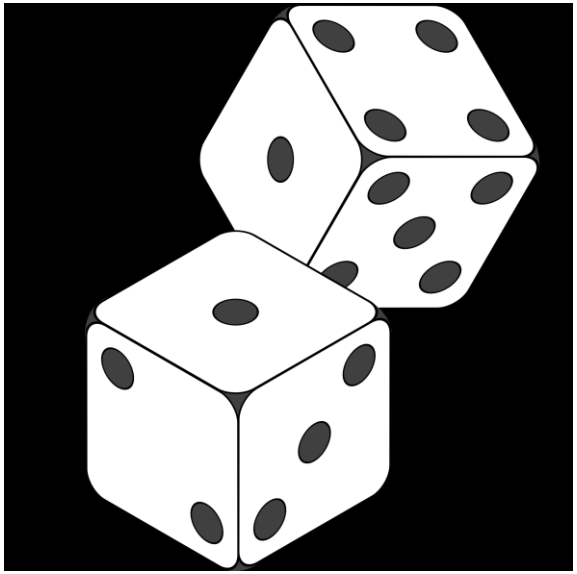
Components of Local Search

Term	Description
Initialisation Method	Local search algorithms typically work by perturbing an initial solution. How is that solution generated?
Neighbourhood Function	In local search, we only consider trying "nearby" solutions. The neighbourhood function defines which solutions are nearby.
Step Function	The rules determining which solution in the neighbourhood we visit.
Termination Condition	The condition on which to stop the search procedure

- Typically, local search is **iterated**:
  - Start with an initial solution
  - Choose a new solution in the *neighbourhood* according to the *step function*
  - Keep stepping to neighbouring solutions until the *termination condition is reached*

Local Search Considerations: Initialization





- Good quality initial solutions are often a good place to start
- But local search can improve solutions quickly. The more time we spend on the initializer, the less time we have to make improvements
- Determinism? If we always start from the same place our final result is unlikely to change much

Local Search Considerations: Neighbourhood

- What do we want in a good neighbourhood?
  - It should contain **feasible solutions**
  - The neighbours of a solutions should be **“similar”** to it
    - What do we mean by similar?
    - Good rule of thumb: similar in cost/fitness
  - It should be **small** in comparison to problem size
    - No use replacing one unsolvable problem with another!
    - Exactly how small should it be?
      - Very small neighbourhoods are quick to explore…
      - …but are more prone to getting trapped in **local optima**



Local Search Considerations: Step Function



- We have a neighbourhood. How should we move?
- Sample strategies:
  - **At random**: quick to evaluate, slow to improve, can't get trapped
  - **Best neighbour**: slow to evaluate, quick to improve
  - **Better neighbour**: quicker to evaluate, slower to improve
  - **Probabilistic**:
    - depends on the details
    - performance can be between best and better neighbour
    - can avoid traps
- How to pick a strategy?
  - Reasonable approach: if you have a bigger neighbourhood, choose a quicker step function

"Hill climbing"

Local Search Considerations: Termination



- Terminate if:
  - you have used all your resources (time's up!)
  - you have achieved your aim
  - you cannot make any further improvement

- If you have resources available, does the final option make sense?
  - random restart is a powerful technique

How Can I Apply Local Search to the TSP?



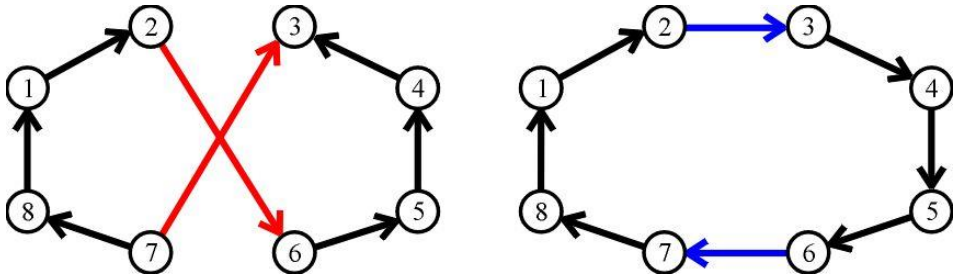
A Local Search Algorithm for the TSP

- Recall that, to define a local search algorithm, we need:
  - **Initialization Method**
  - **Neighbourhood Function**
  - **Step Function**
  - **Termination Condition**
- We will assume that we have very limited time
- Initialization is easy: use one of our constructive heuristics
  - We will use the greedily constructed tour:
    - quick to construct
    - reasonable solution quality

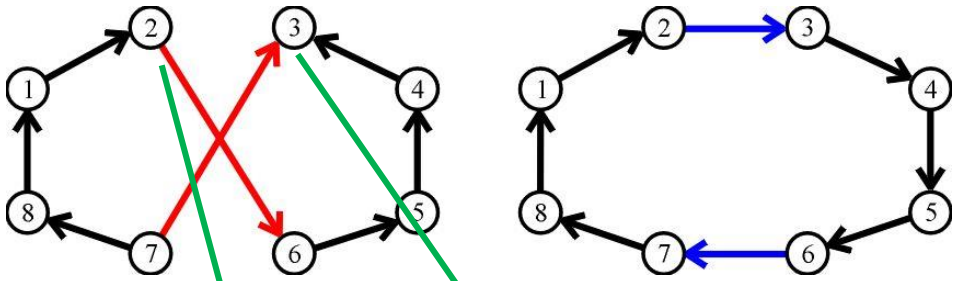


Neighbourhood Choice

- What makes two TSP tours “similar”?
  - Sharing edges
  - How many edges can two distinct, valid tours share?
    - all but two
    - Good candidate for neighbourhood: set of tours sharing all but two edges.
    - Known as **2-opt**



2-opt swap implementation



- How to implement?
  - Identify the two (non-adjacent) cities BEFORE the edges to be swapped



  - Add the part of the old route up until the first city to the new route



  - Add the part of the old route between the two cities to the new route **in reverse order**



  - Add the remaining cities



Step Function

- We are assuming that we have limited time
- 2-opt neighbourhood is reasonably small
- Can use “best improver” as a step function

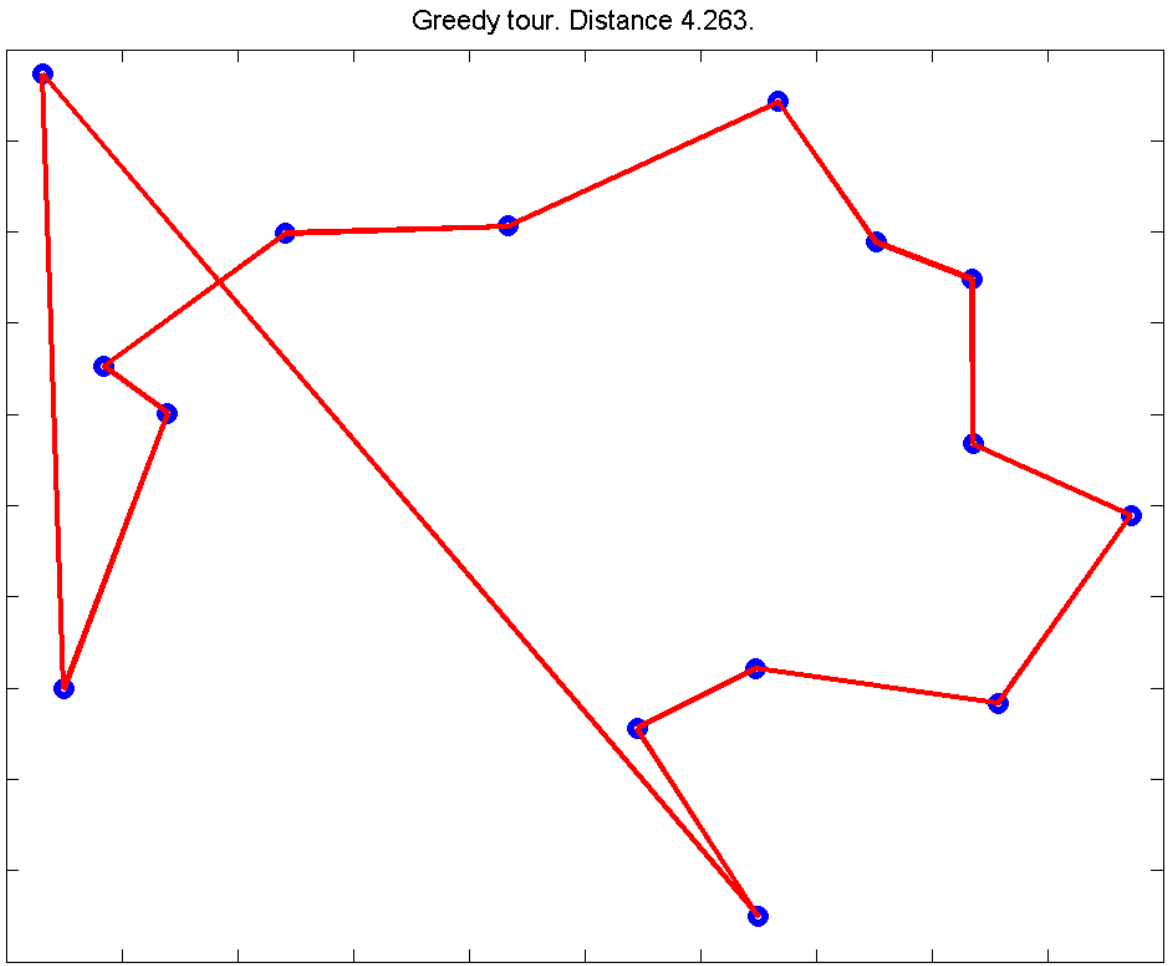


Termination

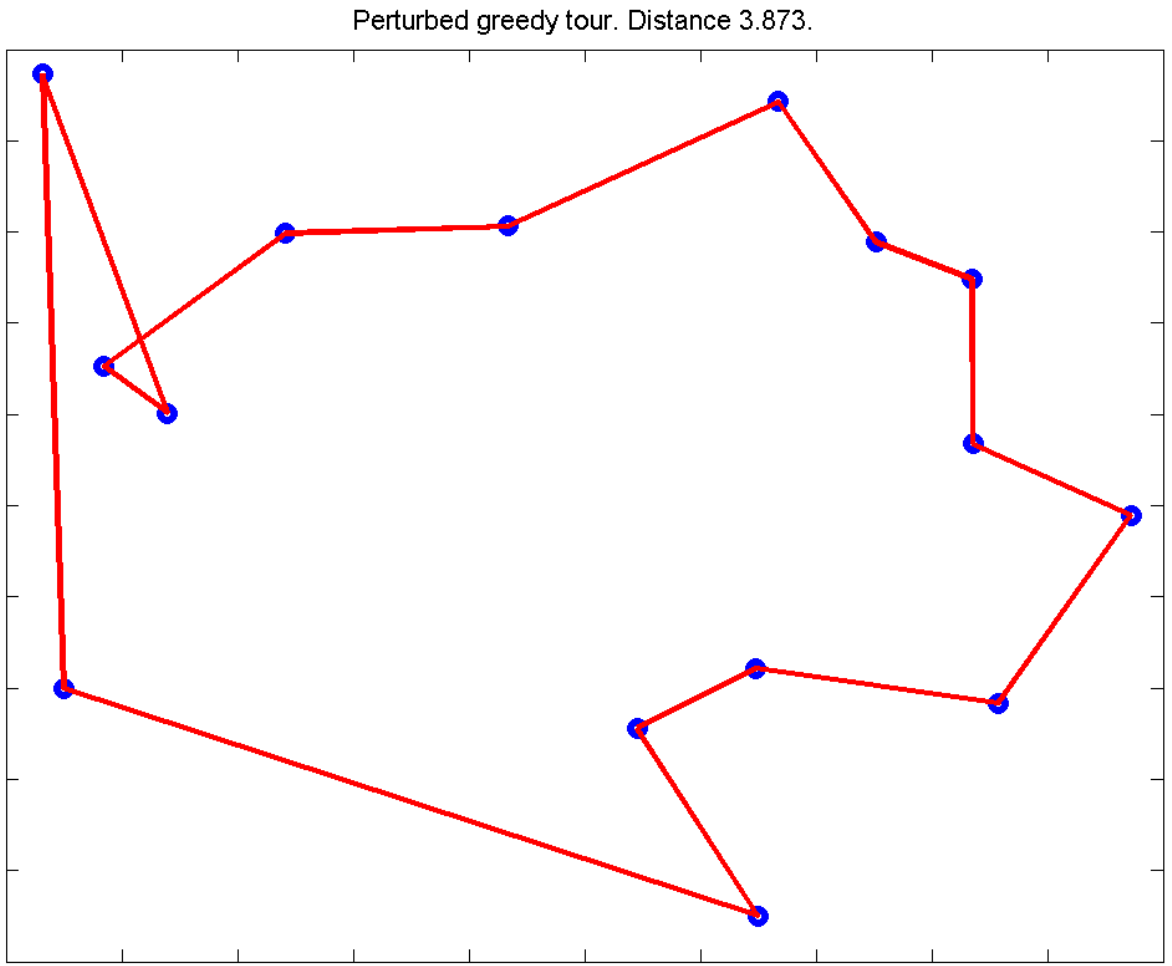
- For simplicity, we will stop when a local minimum is found
- In practice, we could randomly restart until our time budget has been used up



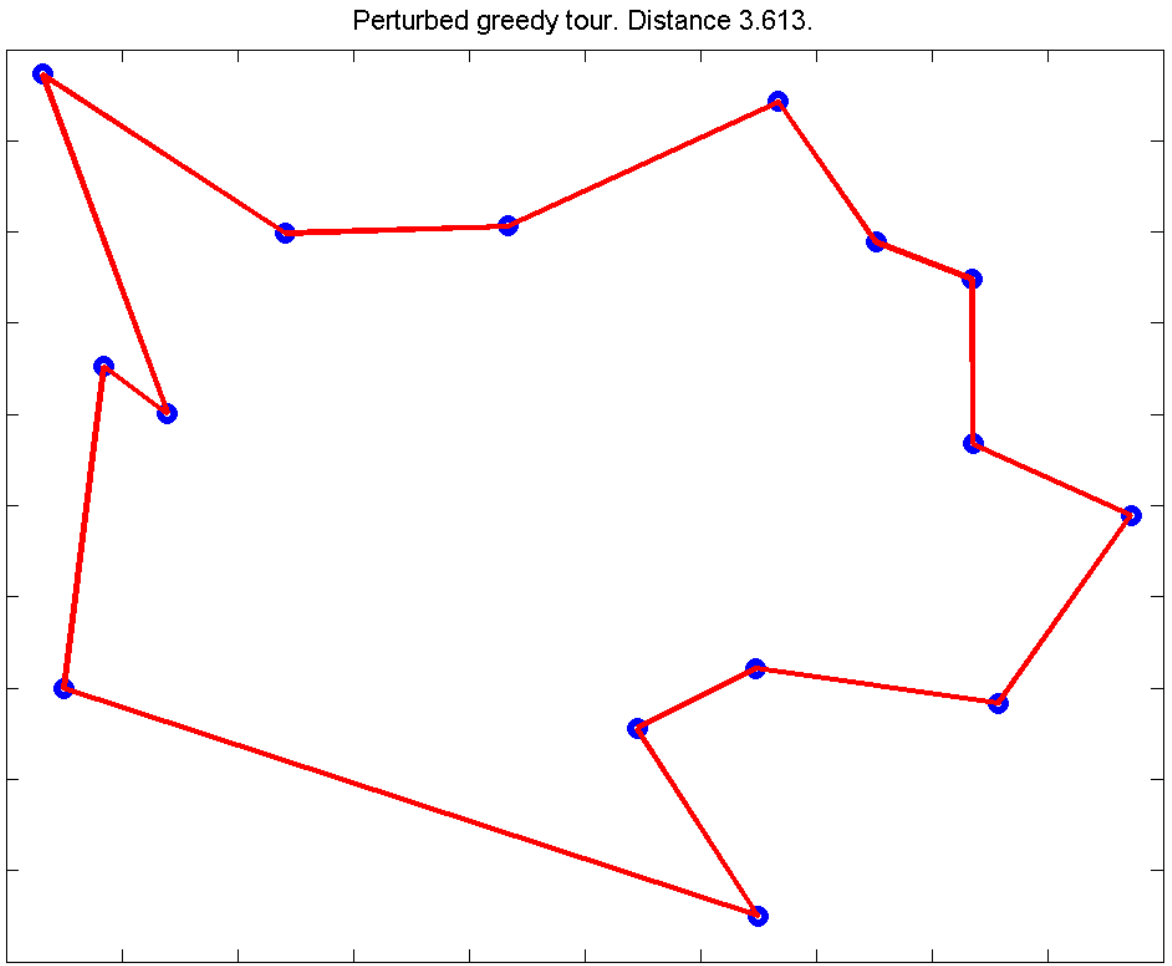
Greedy Tour



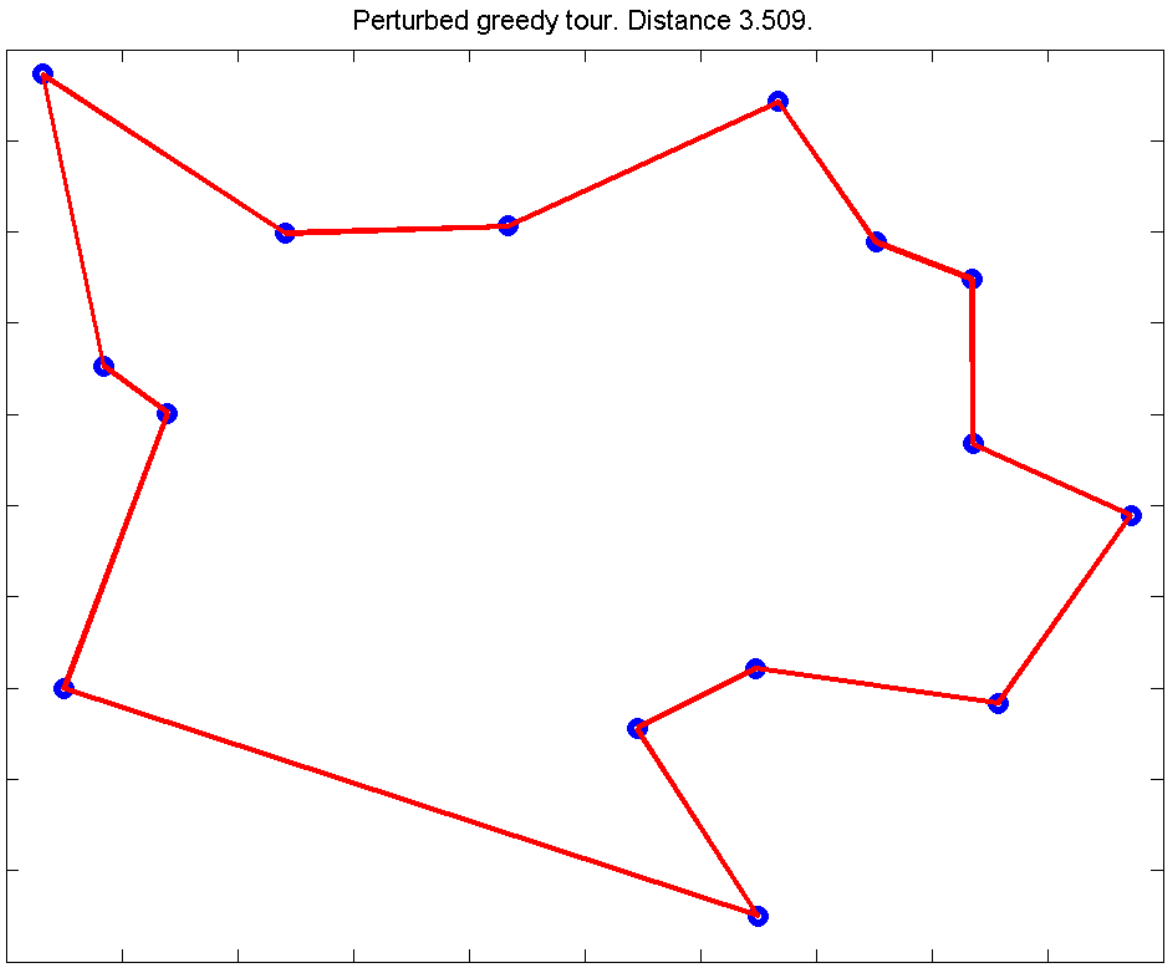
Improving on the Greedy Tour (1)



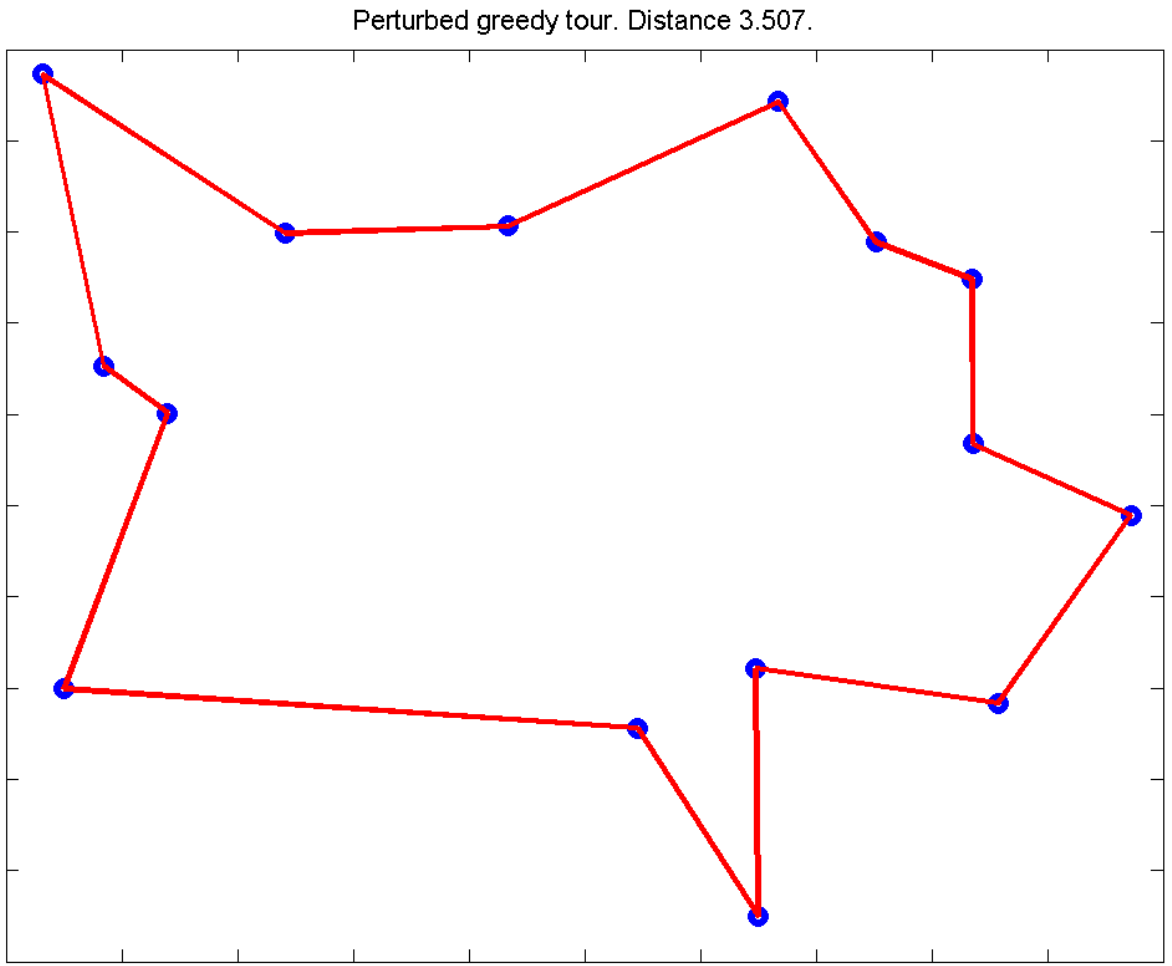
Improving on the Greedy Tour (2)



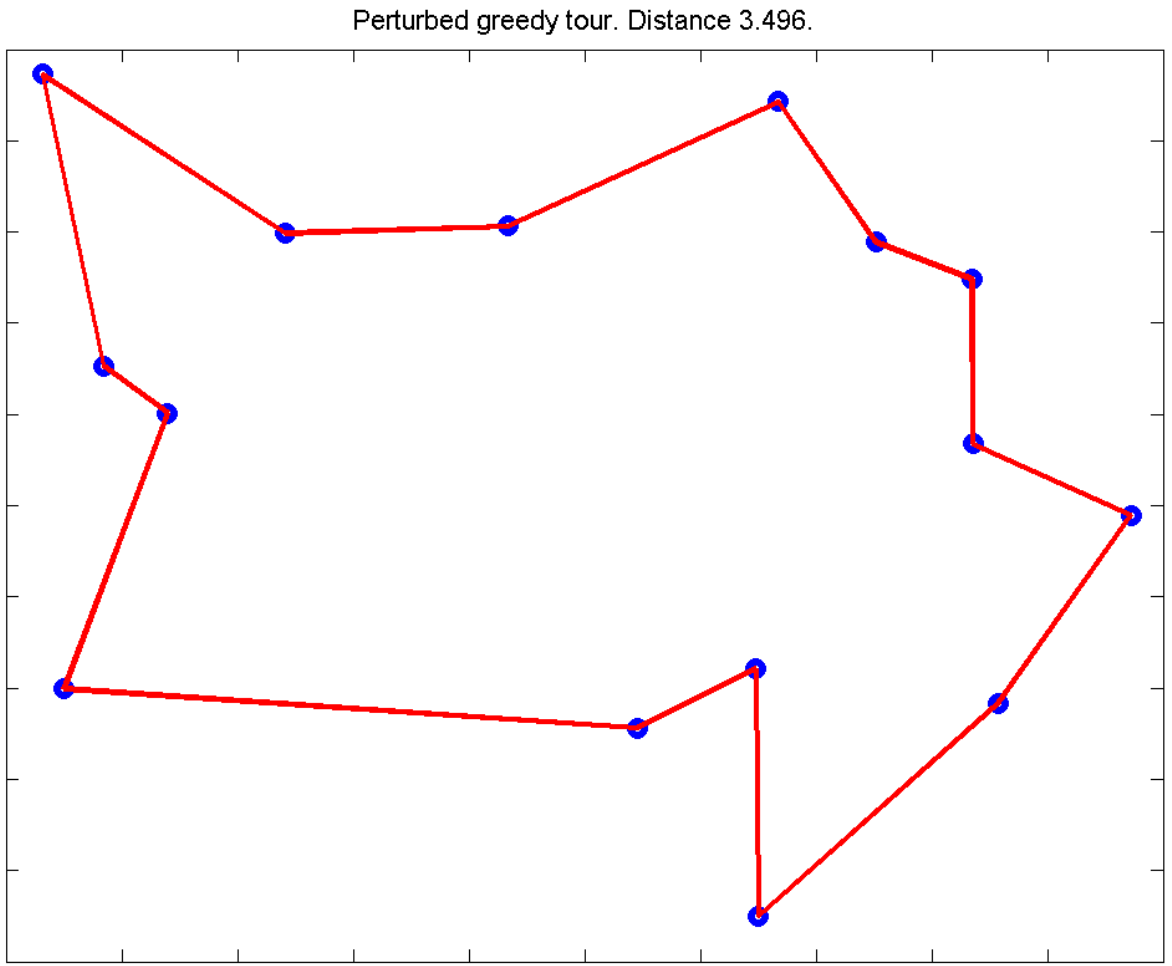
Improving on the Greedy Tour (3)



Improving on the Greedy Tour (4)



Improving on the Greedy Tour (5)



How can I apply local search to a continuous space?



Real-valued Optimization

- Key difference between **discrete optimisation** (e.g. TSP) and **real-valued/continuous** optimisation (e.g. antenna design):
  - reasonably chosen neighbourhoods contain an **infinite number of solutions**
  - cannot search systematically
  - cannot even search the neighbourhood systematically
    - Step functions like “best improver” not possible
- However, the basic ideas still apply



A Simple Real-valued Optimizer

- Generate a solution at random (*current candidate*)
- Until a termination condition is reached:
  - Generate a *new candidate* from a normal distribution centred on the *current candidate*
  - Evaluate the *new candidate*
  - Replace the *current candidate* with the *new candidate* if it is better
- Return the *current candidate*



Summary



Summary

- At the end of this lecture you should…
  - …be able to explain why it is important to have more than one solution to a problem
  - …understand the principles of designing a local search algorithm
  - …understand two given discrete and real valued local search algorithms well enough to implement them.



Recommended Reading

- Books
  - Hoos and Stützle: *Stochastic Search: Foundations and Applications*, Chapter 1, 2.1 and 2.2
- Papers
  - Lourenço, Martin and Stützle : *Iterated Local Search*
  - Beyer and Schwefel: *Evolution Strategies, A Comprehensive Introduction*

